DBMS TASK 8

BrightMartDB – Query Outputs & Business Reports

Database	BrightMartDB
SQL source	dbms task 8.sql
Report sections	Query Output Screenshots • Product Search • Customer Information • Business Analysis

Prepared from the SQL statements supplied for Task 8. Actual result values and execution screenshots must be captured from the BrightMartDB database; they are not fabricated in this report.

1. SQL Task Overview

The supplied SQL script uses INNER JOIN, LEFT JOIN, RIGHT JOIN, filtering for unmatched records, aggregate functions, grouping, sorting and LIMIT. The statements examine relationships among Customer, Orders, Order_Details, Product and Payment.

Query groups

Query range	Purpose
1–3	Customer–order, order–payment and customer–product relationships using INNER JOIN.
4–6	LEFT JOIN results, including customers without orders and products never ordered.
7–9	RIGHT JOIN results and records with no matching customer.
10–12	Combined order/payment detail and customer–product listing.
13–18	Customer spending, order count, latest purchase, order status, product sales, payment summary and top-five ra

Execution note: Run the uploaded *dbms task 8.sql* in MySQL after confirming that BrightMartDB and its tables/data are available. The script begins with `USE BrightMartDB;`.

2. Query Output Screenshots

Insert screenshots from MySQL Workbench (or the SQL client used) after executing the statements. Each screenshot should show the query and its result grid clearly. The SQL file contains 18 SELECT statements; group related results into readable screenshots if needed.

Screenshot	Queries / output to capture
01	Queries 1–3: customer/order, order/payment and customer/product joins.
02	Queries 4–6: left join results, customers without orders, products never ordered.
03	Queries 7–9: right join results and unmatched customer records.
04	Queries 10–12: combined order/product/payment details and customer-product list.
05	Queries 13–15: customer total spent, total orders and latest purchase.
06	Query 16: customer order history and order status.
07	Query 17: product quantity sold and revenue.
08	Query 18: payment-mode transaction summary and top-five reports (see remaining statements in SQL file).

Screenshot checklist: include the executed SQL statement, column headings, all visible result rows (or a clear indication of row count), and no error messages.

3. Product Search Report

Objective: identify products that have been purchased and summarize sales quantity and revenue.

The product-sales query joins Product with Order_Details using Product_ID, then groups by Product_Name.

```
SELECT Product.Product_Name,
       SUM(Order_Details.Quantity) AS Quantity_Sold,
       SUM(Order_Details.Quantity * Order_Details.Price) AS Revenue
FROM Product
JOIN Order_Details
  ON Product.Product_ID = Order_Details.Product_ID
GROUP BY Product.Product_Name;
```

Report fields

Field	Meaning
Product_Name	Name of the product.
Quantity_Sold	Total units recorded in Order_Details.
Revenue	Sum of Quantity × Price for each product.

Interpretation: Products with higher Quantity_Sold have greater unit movement. Revenue indicates the sales value calculated from the order-detail prices. Products absent from Order_Details will not appear because this query uses an INNER JOIN.

Result table: populate this section using the actual query output.

Product Name	Quantity Sold	Revenue
(from SQL output)	—	—
(from SQL output)	—	—
(from SQL output)	—	—

4. Customer Information Report

Objective: present customer purchase activity, spending, order frequency and latest purchase date.

The supplied SQL provides the following customer-focused views:

Query	Information produced
Customer + product join	Customer_Name and Product_Name for customers with matching orders and order details.
Total spent	SUM(Orders.Total_Amount), grouped by Customer_Name.
Total orders	COUNT(Orders.Order_ID), grouped by Customer_Name.
Latest purchase	MAX(Orders.Order_Date), grouped by Customer_Name.
Order history	Customer_Name, Order_ID, Order_Date and Order_Status.
Top five spenders	Customers ranked by Total_Spending in descending order.
Top five by order count	Customers ranked by Total_Orders in descending order.

Customer report summary

Use the query outputs to identify customers with the highest spending, customers placing the most orders, and each customer's most recent purchase. The order-history query also provides the order status for reviewing customer activity.

Important: the SQL groups by Customer_Name. If two customer records share the same name, their values may be combined in these grouped reports.

5. Basic Business Analysis Report

Objective: use order, product and payment data to understand sales performance and transaction activity.

A. Product performance

The product summary calculates total quantity sold and revenue per product. This can help compare product movement and sales contribution.

B. Payment analysis

```
SELECT Payment_Mode,
       COUNT(*) AS Total_Transactions,
       SUM(Payment_Status = 'Success') AS Successful_Payments
FROM Payment
GROUP BY Payment_Mode;
```

This query groups transactions by Payment_Mode, counts all transactions and counts rows whose Payment_Status is 'Success'.

C. Customer ranking

The final two ranking queries return the top five customers by total spending and by number of orders, respectively. They sort the aggregate values in descending order and use LIMIT 5.

Business interpretation guide

Measure	How to interpret the result
Product quantity sold	Highlights products with the greatest unit sales.
Product revenue	Shows the value generated by each product based on order-detail quantity and price.
Payment transactions	Compares transaction volume across payment modes.
Successful payments	Shows the count of transactions marked Success for each payment mode.
Top spending customers	Identifies the five customers with the largest recorded order totals.
Top customers by orders	Identifies the five customers with the most recorded orders.

Conclusion

Task 8 demonstrates how SQL joins combine related business records and how aggregate functions summarize customer, product and payment activity. Insert the real MySQL outputs and screenshots before submission so the report reflects the actual BrightMartDB data.

Submission checklist

- ■ Include the original dbms task 8.sql file.
- ■ Execute all statements successfully in BrightMartDB.
- ■ Add query output screenshots in Section 2 or as separate pages.
- ■ Replace the placeholder result rows with actual database values.
- ■ Submit this completed PDF report.